

Lesson 18 — Model Deployment & Monitoring

Now we're moving from building ML models to making them useful in a real application.

So far:

```
Data → Train → Evaluate → Tune → Pipeline
```

But a trained model sitting inside a notebook isn't very useful.

We need to deploy it so another application can send data and receive predictions.

1. What is Model Deployment?

Model deployment means making a trained ML model available for real-world predictions.

For example, you train a model that predicts whether a service will fail.

Your model might receive:

```
CPU = 92  
Memory = 88  
Latency = 450  
Error Rate = 7
```

and return:

```
Failure Probability = 0.91
```

Deployment allows another application to request that prediction.

2. Training vs Prediction

This distinction is extremely important.

Training

Usually happens offline:

```
Historical Data  
↓  
Feature Engineering  
↓  
Preprocessing  
↓  
Train Model  
↓  
Evaluate  
↓  
Save Model
```

Prediction

Happens when the application is running:

```
New Data
↓
Same preprocessing
↓
Saved Model
↓
Prediction
```

You normally don't retrain the model every time someone requests a prediction.

3. Basic Architecture

A deployed ML system can look like:

```
User / Application
↓
API
↓
Feature Processing
↓
Trained Model
↓
Prediction
↓
API Response
```

For your SentinelAI project:

```
Prometheus
↓
Telemetry
↓
Feature Engineering
↓
ML Model
↓
Failure Probability
↓
FastAPI
↓
Dashboard
```

4. What is an API?

An API allows different software components to communicate.

For example:

```
Frontend
↓
POST /predict
↓
FastAPI
↓
ML Model
↓
Prediction
↓
JSON response
```

The frontend doesn't need to know how the model works internally.

It simply sends data.

5. Example Prediction API

Let's create a very simple FastAPI application.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class InputData(BaseModel):
    cpu: float
    memory: float
    latency: float
    error_rate: float

@app.post("/predict")
def predict(data: InputData):

    # Example prediction logic
    if data.cpu > 80 and data.error_rate > 5:
        prediction = 1
    else:
        prediction = 0

    return {
        "prediction": prediction
    }
```

This isn't a real ML model yet—we're just understanding deployment.

6. What Does `BaseModel` Do?

```
class InputData(BaseModel):
    cpu: float
    memory: float
    latency: float
    error_rate: float
```

It defines what the API expects.

For example:

```
{
  "cpu": 92,
  "memory": 88,
  "latency": 450,
  "error_rate": 7
}
```

FastAPI validates this input.

7. Calling the API

You could send:

```
POST /predict
```

with:

```
{
  "cpu": 92,
  "memory": 88,
  "latency": 450,
  "error_rate": 7
}
```

and receive:

```
{
  "prediction": 1
}
```

This is the basic idea behind deploying an ML model.

8. Real ML Model Deployment

Now imagine we've already trained a model:

```
model.fit(X_train, y_train)
```

We save it:

```
import joblib

joblib.dump(model, "model.pkl")
```

Now we have:

```
model.pkl
```

This file contains the trained model.

9. Load the Model in the API

When the server starts:

```
import joblib

model = joblib.load("model.pkl")
```

Then:

```
@app.post("/predict")
def predict(data: InputData):

    X = [[
        data.cpu,
        data.memory,
        data.latency,
        data.error_rate
    ]]
```

```

prediction = model.predict(X)

return {
    "prediction": int(prediction[0])
}

```

Now the API is using an actual trained ML model.

10. What About Scaling?

Remember Lesson 17?

Suppose training used:

```

StandardScaler
  ↓
Model

```

You must use the same preprocessing during prediction.

That's why using a pipeline is so useful.

Instead of saving only:

```
model.pkl
```

we can save:

```

Pipeline
├── StandardScaler
└── Model

```

Example:

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipeline.fit(X_train, y_train)

```

Then save the whole pipeline:

```

import joblib

joblib.dump(pipeline, "model.pkl")

```

Now deployment becomes much safer:

```

New Data
  ↓
Saved Pipeline
  ↓
Scaler
  ↓
Model
  ↓

```

11. Model Monitoring

Deployment isn't the end.

Once your model is running in production, you need to monitor it.

Why?

Because the real world changes.

For example, your model was trained on:

```
CPU: 0-100%  
Memory: 0-100%  
Latency: 10-500 ms
```

Six months later, your system might behave differently.

The data distribution can change.

12. What is Data Drift?

Data drift means the distribution of input data changes over time.

Example:

During training:

```
Average CPU = 45%
```

Production later:

```
Average CPU = 80%
```

The model may start receiving data unlike what it saw during training.

This can reduce performance.

13. What is Concept Drift?

Concept drift is different.

The relationship between inputs and the target changes.

Example:

Previously:

```
CPU > 90%  
→ high probability of failure
```

But after a major infrastructure upgrade:

```
CPU > 90%
```

→ completely normal

The meaning of the data has changed.

14. What Should We Monitor?

A production ML system can monitor:

Model performance

Accuracy
Precision
Recall
F1
MSE

depending on the task.

Input data

CPU
Memory
Latency
Error rate

Prediction behavior

Prediction distribution
Failure probabilities
Confidence

System health

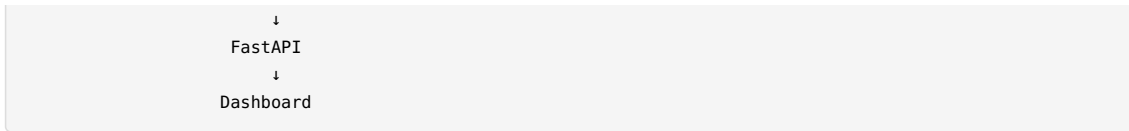
API latency
CPU usage
Memory usage
Request rate
Errors

15. SentinelAI Monitoring 🍄

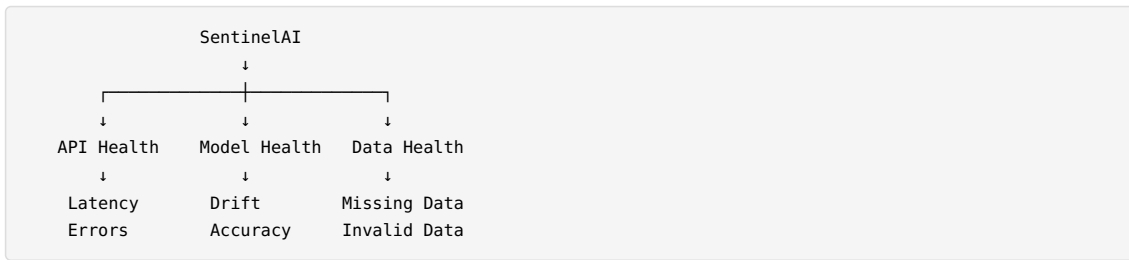
This is where Lesson 18 becomes directly relevant to your project.

Your architecture can eventually look like:

```
graph TD
    PROMETHEUS --> MD[Metric Discovery]
    MD --> MN[Metric Normalization]
    MN --> FE[Feature Engineering]
    FE --> FA[Feature Availability]
    FA --> MLP[ML Prediction]
    MLP --> FP[Failure Probability]
    FP --> ES[Explanation/Severity]
```



And the system itself can be monitored:



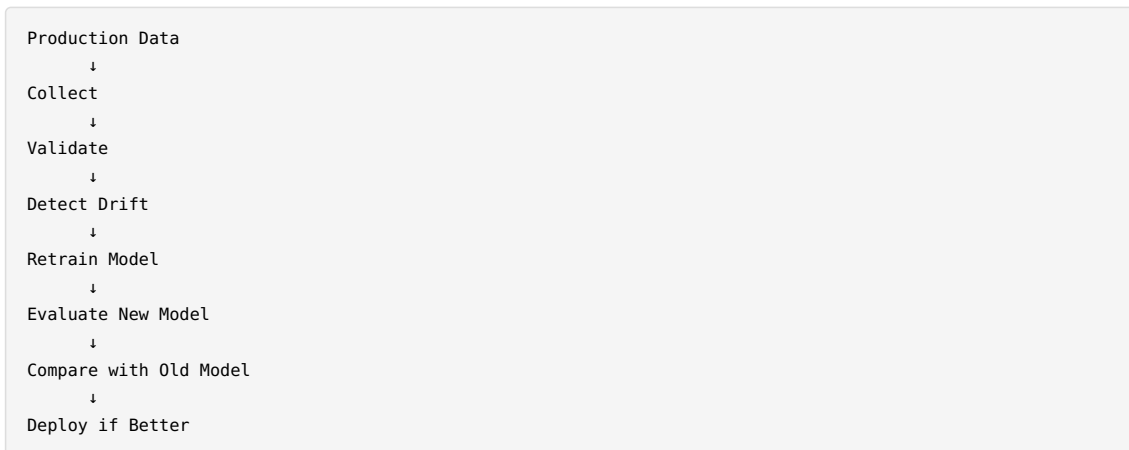
This is much closer to production ML / MLOps.

16. Model Retraining

Suppose your model becomes less accurate.

You may need to retrain it.

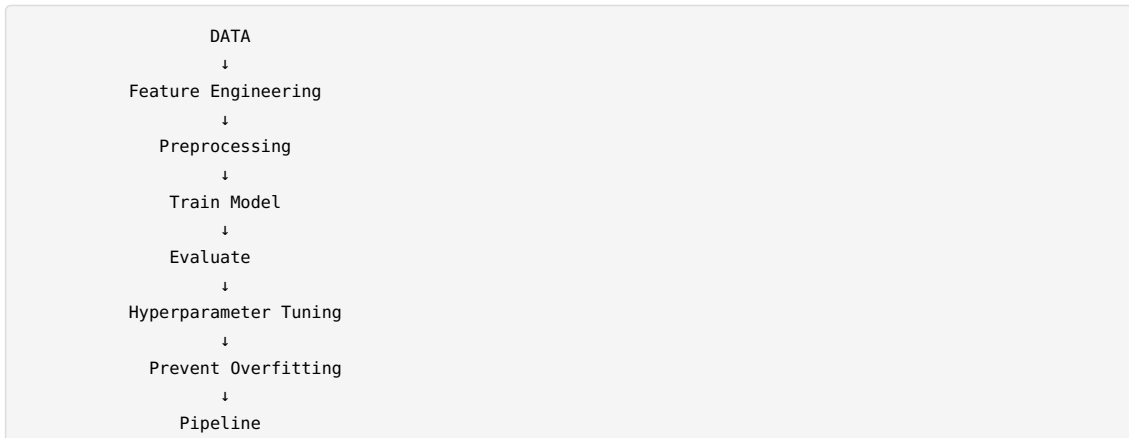
Typical workflow:

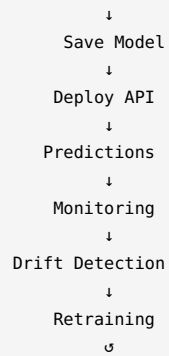


This creates a continuous ML lifecycle.

🧠 17. The Complete ML Lifecycle

You've now reached an important point in the roadmap.





That's essentially the ML lifecycle.

What You Should Remember

Deployment

Making a trained model available for real-world predictions.

API

Allows applications to communicate with the model.

Model serialization

Saving a trained model so it can be loaded later.

Data drift

Input data distribution changes.

Concept drift

The relationship between input and target changes.

Monitoring

Continuously checking model, data, and system behavior.

Retraining

Training a new model when the existing model no longer performs adequately.